

Mediterranean Institute of Technology

Computer System Engineering Program

Data Science Project Report

Course Code: CS434 – Data Science

Cross-Network Intrusion Analysis

Detecting Cyberattacks Across Five Network Environments with CRISP-DM

Submitted By:

Ahmed Hamouda
Cyrine Chabbar
Yassine Mtibaa

Professor:

Dorra Louati

Lab Instructor:

Hamza Mouharrar

Date: October 2026

Abstract

Machine-learning intrusion detection systems are usually built and tested on a single network, so their reliability on another network is unknown. This project follows the CRISP-DM process, from business understanding to feature engineering, to prepare and analyse traffic from **five network environments**. The data are four NetFlow v3 datasets (UNSW-NB15, CSE-CIC-IDS2018, ToN-IoT, BoT-IoT) and CICIoT2023, mapped to one shared taxonomy of 9 attack families.

Preparing the data. A systematic quality check found mislabelled traffic, columns broken by divisions by zero, and duplicated captures. After the cleaning, the four NetFlow datasets were merged into one table of **17,551,848 distinct behaviours representing 63,012,224 flows**. CICIoT2023 was kept separate (20,701,050 clean packet windows), because a row there measures something different.

What the analysis shows.

1. No traffic variable is normally distributed, so non-parametric tests are used.
2. Attack and normal traffic differ strongly inside each network, but *in opposite directions* in different networks: attacks are smaller than normal traffic in UNSW-NB15 and larger in ToN-IoT.
3. The same attack family does not produce the same traffic in different networks (Kolmogorov–Smirnov distance up to 0.99), nor in a different dataset.

Feature engineering produced a justified set of **59 features** for the NetFlow data, with every identifier of the network or the lab removed, and **31 features** for CICIoT2023. These results point to a clear expectation for the next step: a detector trained on some networks will lose accuracy on a new network, and the modelling plan proposed at the end is designed to measure that loss.

Contents

Abstract **1**

1 Business Understanding **3**
1.1 Project context . . . 3
1.2 Problem statement . . . 3
1.3 Research questions . . . 3
1.4 Success criteria . . . 4

2 Data Understanding **5**
2.1 Data sources and provenance . . . 5
2.2 Dimensions and structure . . . 5
2.3 Variables, data types and units . . . 5
2.4 A shared attack taxonomy . . . 6
2.5 Initial quality issues and limitations . . . 7

3 Data Preprocessing **9**
3.1 Principles . . . 9
3.2 Cleaning each NetFlow dataset . . . 9
3.3 Integration: merging the four NetFlow datasets . . . 10
3.4 Cleaning CICIoT2023, and why it stays separate . . . 11
3.5 Preventing data leakage . . . 11

4 Exploratory Data Analysis **13**
4.1 Descriptive statistics . . . 13
4.2 Distributions . . . 14
4.3 Correlation analysis . . . 15
4.4 Relationships between variables . . . 17
4.5 Normality and the choice of tests . . . 18
4.6 Hypothesis tests . . . 19
4.7 The same attack in a different dataset: CICIoT2023 vs NetFlow . . . 21
4.8 Summary of the analysis . . . 22

5 Feature Engineering **23**
5.1 Approach . . . 23
5.2 Creation / extraction . . . 23
5.3 Encoding . . . 24
5.4 Transformation and scaling . . . 25
5.5 Feature selection . . . 26
5.6 Final feature sets and how they support the objectives . . . 28

6 Conclusions and Next Steps **30**
6.1 Answers to the research questions . . . 30
6.2 Limitations . . . 30
6.3 Future modelling plan (not implemented) . . . 31

References **32**

A Reproducibility **33**

B Feature dictionaries and documentation **34**

C Team contributions **35**

1 Business Understanding

1.1 Project context

Every organisation connected to the internet is a target: its network is probed by scans, flooded by denial-of-service attacks, attacked with password guessing, and infected by malware that turns its devices into bots. **Network Intrusion Detection Systems (NIDS)** monitor the traffic and raise an alert when it looks malicious. Security teams face two costly failures: **missed attacks**, and **false alarms** that waste analysts' time until real alerts are ignored.

Modern NIDS increasingly use machine learning, learning what attacks look like from labelled traffic. But organisations run very different networks (offices, industrial IoT, smart devices), and a detector trained on one kind of network often performs much worse on another. An organisation usually cannot collect and label its own attack data before deploying a detector.

1.2 Problem statement

> Organisations need an intrusion detection approach that recognises cyberattacks accurately, tells the security team **what kind** of attack is happening, and stays reliable on a **different type of network** from the one it was trained on.

As a data science problem:

1. integrate traffic recorded on five different networks into consistent, analysis-ready data;
2. organise all attack types into one shared taxonomy;
3. analyse which traffic characteristics reveal each attack family, and whether they hold from one network to another;
4. prepare the features for a future classifier.

1.3 Research questions

Main question: *How can traffic collected from different network environments be used to detect cyberattacks and identify their type reliably, including on networks the detection model has never seen?* It is broken into three sub-questions (Table 1).

Table 1: Sub-questions. SQ1 identifies *what* reveals an attack, SQ2 tests *whether* it transfers between networks, SQ3 asks *how well* a model could use it.

SQ1	Which traffic characteristics best distinguish normal traffic from attack traffic, and how do they differ across attack families?
SQ2	How consistent is the traffic behaviour of the same attack type across different network environments?
SQ3	How accurately could a model detect attacks and identify their family, and how much would its performance change on a network it was not trained on?

1.4 Success criteria

The project is successful if:

1. **every sub-question is answered with evidence**: statistical tests, effect sizes and figures, not assumptions;
2. **the key indicators of each attack family are identified**: we can explain what gives each type of attack away;
3. **the data are model-ready and free of leakage**: a justified final feature set with no column that identifies the network, the machine or the moment;
4. **the work is understandable and reproducible**: every decision is documented, and anyone can rebuild the results from the repository.

Model training is outside the scope of this project; SQ3 is answered as an evidence-based expectation and a modelling plan (Section 6).

2 Data Understanding

2.1 Data sources and provenance

Five public datasets cover five network environments. The four NetFlow v3 datasets were re-extracted by the University of Queensland from the original packet captures with the same tool (nProbe) into the same 53 features, so a column means exactly the same thing in all four files [1, 2]. CICIoT2023 was recorded on 105 real IoT devices with a different feature extractor [7]. Table 2 summarises the five datasets.

Table 2: The five datasets. Flows and attack shares are counted in the loaded files.

Dataset	Network environment	Rows	Attack %	Attack labels	Owner
NF-UNSW-NB15-v3 [3]	Mixed lab network	2,365,424	5.4	9	A. Hamouda
NF-CSE-CIC-IDS2018-v3 [4]	Enterprise / office	20,115,529	12.9	14	A. Hamouda
NF-ToN-IoT-v3 [5]	IoT / industrial	27,520,260	39.0	9	C. Chabbar
NF-BoT-IoT-v3 [6]	IoT botnet	16,933,808	99.7	4	C. Chabbar
CICIoT2023 [7]	105 real IoT devices	45,019,243*	97.7	33	Y. Mtibaa

* packet windows, not flows.

2.2 Dimensions and structure

- **NetFlow v3:** each row is **one flow** (a connection from its first to its last packet), with 53 features plus `Label` (0/1) and `Attack` (class name). The loaded row counts match the official numbers except BoT-IoT. Its page states 16,993,808 rows, but the file has 16,933,808, which is also the sum of its own Benign and attack counts: a typo on the official page.
- **CICIoT2023:** each row is **one window of up to 100 packets**, described by 39 statistics (shares of packets per protocol and flag, packet-size statistics, rate) plus `Label`. We used the newer `MERGED_CSV` release (63 files, 9.3 GB). It differs from the 2023 documentation: 45,019,243 rows instead of 46.7 million, and 8 columns removed or renamed.

2.3 Variables, data types and units

Table 3 groups the NetFlow variables by role, with their type and unit.

Key variables. The volume, packet-size and timing columns describe *what the traffic does*; the destination port, protocol and TCP flags describe *how*. For CICIoT2023 the key variables are `Rate` (packets/s), the packet-size statistics (`AVG`, `Std`, `Max`), the protocol and flag shares, `IAT` and `Header_Length`.

Table 3: NetFlow v3 variables by group. The full dictionary of the 53 features (meaning, unit, type and what happened to each) is in Appendix B.

Group	Example columns	Type	Unit
Identifiers	`IPV4_SRC_ADDR`, `L4_SRC_PORT`, `DNS_QUERY_ID`	text / code	–
Protocol / service	`PROTOCOL`, `L4_DST_PORT`, `L7_PROTO`	code	–
Volume	`IN_BYTES`, `OUT_BYTES`, `IN_PKTS`, `OUT_PKTS`	integer	bytes, packets
Packet sizes	`LONGEST_FLOW_PKT`, `NUM_PKTS_UP_TO_128_BYTES`	integer	bytes, packets
Time	`FLOW_DURATION_MILLISECONDS`, `SRC_TO_DST_IAT_AVG`	integer	ms
Timestamps	`FLOW_START_MILLISECONDS`	integer	ms since 1970
TCP behaviour	`CLIENT_TCP_FLAGS` (bitmask), `TCP_WIN_MAX_IN`	code / integer	–, bytes
Labels	`Label`, `Attack`	integer / text	–

2.4 A shared attack taxonomy

The five datasets name their attacks differently (54 labels, often with different spellings), so every label was mapped to one of **9 attack families**: Denial of service, Reconnaissance, Credential attacks, Web and injection, Malware and botnets, Exploitation and intrusion, Interception and spoofing, Fuzzing and crypto, Data theft. The mapping is fixed in the shared code (`src/config.py`), so every dataset uses the same families.

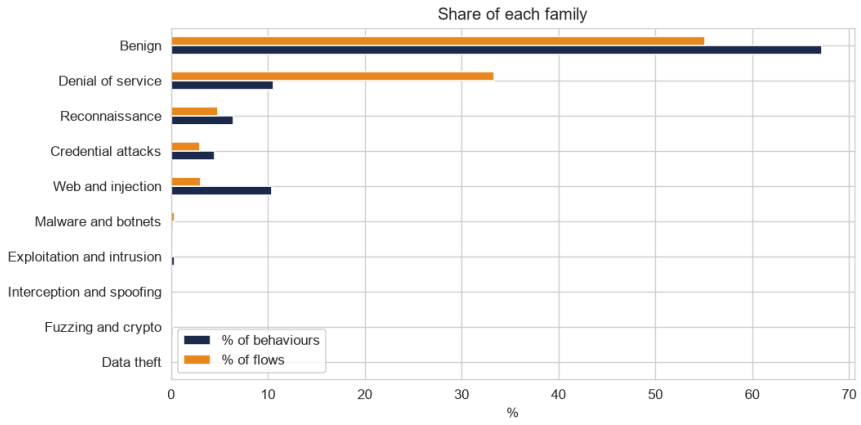


Figure 1: Share of each family in the merged NetFlow data, counted in distinct behaviours and in flows (after cleaning).

Interpretation. Both data sources are strongly imbalanced, but in different ways (Figures 1 and 2). In NetFlow, Benign dominates the distinct behaviours (67%), while Denial of service dominates the flows (33%) because a flood repeats the same flow millions of times. CICIoT2023 is 97.7% attacks, mostly floods. Not every family exists in every network: only Benign and Denial of service are present in all four NetFlow networks, which limits the comparisons across networks (SQ2).

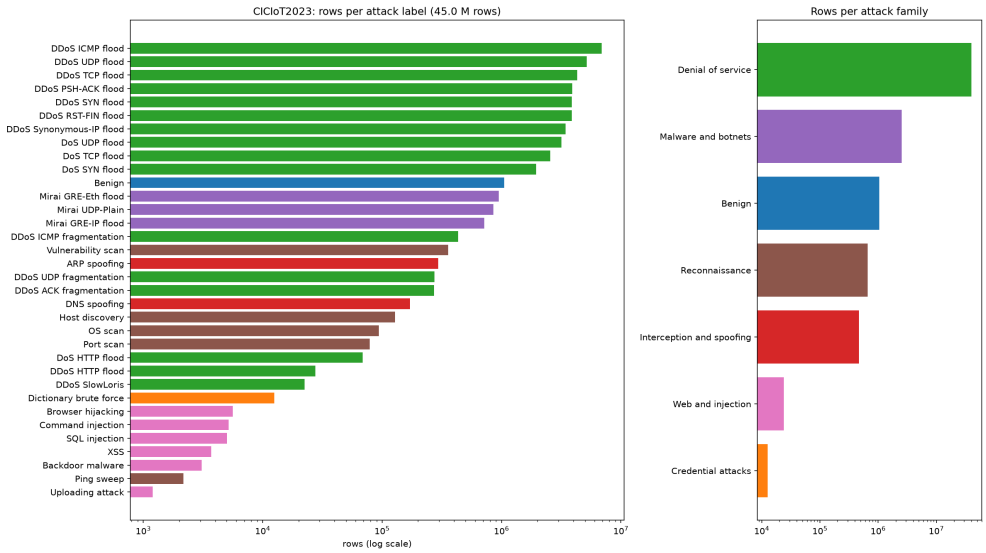


Figure 2: CICIoT2023: number of rows per attack label before sampling (log scale). The largest label is 5,764 times the smallest.

2.5 Initial quality issues and limitations

Each dataset went through the same checks, following the course lab: structure, missing and infinite values, data types, outliers (IQR, z-score, box plots), consistency (duplicates, identifiers, logical rules) and class distribution. For NetFlow, every problem was documented with evidence in a dedicated report, split into problems solved before the merge (codes U, C, T, B) and after it (M), and findings kept on purpose (K). Table 4 lists the most important ones.

Table 4: The most important quality issues found (phase 2, diagnosis only).

Issue	Evidence	Dataset
NaN / infinite rates	bytes ÷ 0 on 0-ms flows: 387,350 NaN + 1,716,860 inf (65% of Benign)	BoT-IoT, UNSW
Two codes in one number	`L7_PROTO` = `5.239` (DNS carrying another protocol)	UNSW, BoT-IoT
Duplicated captures	628,474 rows (CIC18), 1,816,137 (ToN); 51% of CICIoT2023	all
Mislabelled traffic	“DoS SlowHTTPTest” flows are refused FTP connections (105,550)	CIC-2018
Labels on normal traffic	150,600 attack flows identical to Benign flows of the same machines	CIC-2018
Web attacks on DNS traffic	1,261,353 “XSS” / “injection” flows to port 53	ToN-IoT
Same flow, two labels	880 rows (NetFlow); 13.3 M rows, mostly DoS vs DDoS (CICIoT2023)	CIC-2018, CICIoT2023
Identifiers that reveal the label	IP addresses, timestamps, TTL, TCP window	all
Documentation errors	BoT-IoT row count; release differs from paper (CICIoT2023)	BoT-IoT, CICIoT2023

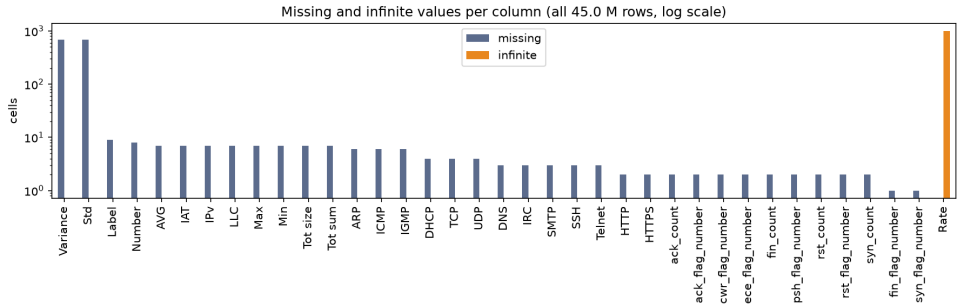


Figure 3: CICIoT2023: missing values per column, caused by 7 truncated lines in the raw files.

Limitations of the data. All five datasets were recorded in testbeds with scripted attacks, so they reflect how each lab was built as well as how attacks behave. Phase 3 and the analysis account for this.

3 Data Preprocessing

3.1 Principles

- **One shared code base.** Every member uses the same functions (`src/config.py`: cleaning, label mapping, validation), so differences between networks reflect the traffic, not the preprocessing.
- **Decide first, then apply.** Every treatment was proposed in phase 2, approved by the team, and recorded in the decision log before being applied in phase 3.
- **Evidence before and after.** Each cleaning notebook reports row counts and checks after every step (`tables/*_quality.csv`).

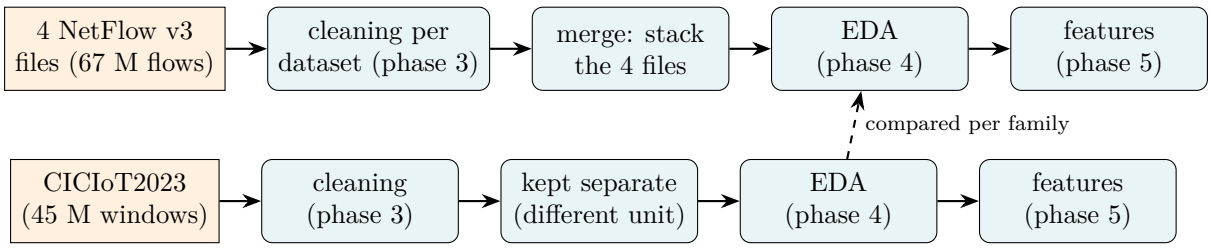


Figure 4: The data pipeline. NetFlow and CICIoT2023 are processed with the same steps; they are compared at the level of attack families, never merged.

3.2 Cleaning each NetFlow dataset

Each NetFlow dataset has its own cleaning notebook, which applies only that dataset's approved decisions, in this order:

1. **Fix broken columns.** The two bytes-per-second columns are dropped (U2, B2, C3, T3). They are NaN or infinite on every 0-ms flow; deleting those rows instead would have removed 65% of BoT-IoT's Benign traffic. `L7_PROTO` is split into two integer codes, `L7_MAIN` and `L7_SUB` (U3, B3).
2. **Remove exact duplicate rows** (U4, C4, T4).
3. **Remove mislabelled traffic** (C5–C7, T5). Flows that carry two labels, SlowHTTPTest flows that are refused FTP connections, attack labels on flows identical to Benign traffic of the same machines, and web attacks labelled on DNS traffic. Relabelling them as Benign was considered and rejected, because it would invent labels.
4. **One row per distinct behaviour.** Identical flows (same behaviour, ignoring IPs, ports and timestamps) are merged into one row with a `count` column. This replaces the planned cap of 200,000 flows per label, which would have kept only 15–40% of the distinct attack behaviours of the large classes.
5. **Standard cleaning and label mapping:** identifiers dropped, labels mapped to the taxonomy, `Label` dropped (identical to `is_attack`).

6. **Validation:** `validate()` reports 0 violations, and there are no NaN or infinite values.

Outliers. Extreme values (huge floods, long connections, packets above 1,514 bytes) were **kept**: in this data the outliers are the attacks. Only impossible values were changed (DNS TTL above $2^{31} - 1$, set to 0 as RFC 2181 prescribes). Table 5 shows the effect of the cleaning on each dataset.

Table 5: Effect of the cleaning on each NetFlow dataset.

Dataset	Raw flows	Clean rows	Flows kept	Main removals
UNSW	2,365,424	1,405,100	2,350,609	14,815 duplicates
CIC-2018	20,115,529	8,945,792	19,285,037	628,474 duplicates; 880 + 105,580 + 95,558 mislabelled
ToN-IoT	27,520,260	5,637,642	24,442,770	1,816,137 duplicates; 1,261,353 web attacks on DNS
BoT-IoT	16,933,808	1,563,315	16,933,808	none (only collapsed into behaviours)

3.3 Integration: merging the four NetFlow datasets

The four cleaned files have the same columns, so they are **stacked** (`pd.concat`), not joined: they contain different flows from different networks and share no key. A column `source_dataset` records the network; like `count`, it is never a model feature. Some decisions need all four datasets together, so they are applied once on the merged table (Table 6).

Table 6: Decisions applied after the merge.

Code	Decision	Effect
M1, M2	codes → categories; timestamps → dates (never features)	10 code columns, 2 timestamps
M6	ICMP fields set to 0 outside ICMP flows	2,879,966 rows corrected
M7, M8	drop columns that copy others (`MIN/MAX_IP_PKT_LEN`, `TCP_FLAGS`, `ICMP_IPV4_TYPE`)	4 columns removed, no information lost
M9	invalid DNS TTL → 0	14 rows; 1 row merged into an identical one
M11	flag behaviours that have a different family in another dataset	38,453 rows flagged, none removed

Result: `netflow_merged.parquet`, **17,551,848 rows representing 63,012,224 flows**, 54 columns, 0 missing values, 0 validation violations.

3.4 Cleaning CICIoT2023, and why it stays separate

- **Treatments.**
  - The 7 truncated lines and the 991 windows with an infinite `Rate` (0-second windows) were removed. No imputation was done: a window with no duration has no meaningful rate.
  - The 2 negative `IAT` values were removed.
  - Exact duplicates were removed per file and again after combining the 63 files.
  - Column names were standardised, and labels mapped to the taxonomy.
  - At most 200,000 rows were kept per label (`sample_per_label`), every row of rarer labels kept.
  - Windows whose features also appear under another label (mostly DoS vs DDoS) were **kept and flagged** (`conflicting_features`), so the finding stays visible in the EDA.
- **Result:** 45,019,243 → 20,701,050 clean windows (24.3 M duplicates removed) → 4,420,714 sampled rows; 0 missing, 0 infinite.
- **Why not merged with NetFlow.** A CICIoT2023 row is a window of up to 100 packets that can mix several connections; a NetFlow row is one connection. The 39 window statistics (shares of packets per flag) are not the same measurements as the 53 flow features (bitmasks per flow). Stacking them would put incompatible values in the same column. The datasets are compared at the level of attack families instead (Section 4.7).

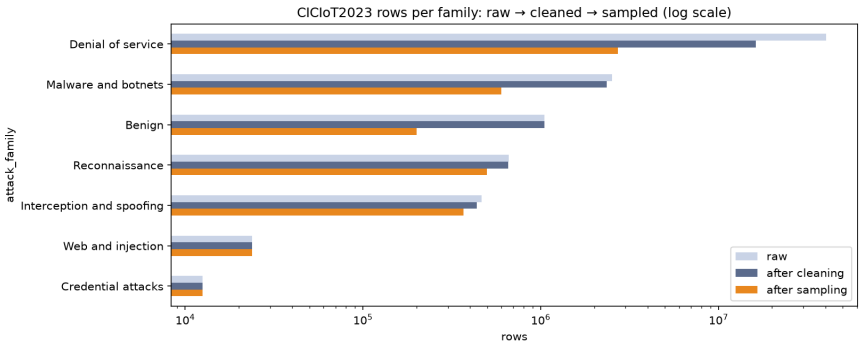


Figure 5: CICIoT2023: rows per family after cleaning and sampling.

3.5 Preventing data leakage

- **Columns that reveal the answer are never features:** IP addresses (they identify the attacker machine), timestamps (they reveal the lab's attack schedule), TTL and TCP window (set by each lab machine's operating system), random identifiers, and `source_dataset` and `count`.
- **Mislabelled flows are removed rather than relabelled,** so that no label is invented.

- **No transformation is fitted per dataset before the analysis.** The project stops before modelling, so there is no train/test split. Every learned step (scaling, selection) must be refitted on the training part only in a modelling phase.

4 Exploratory Data Analysis

The analysis follows the course lab (Lab 4). For NetFlow, every statistic is computed on the **17.5 million distinct behaviours**: identical copies of a flow are not independent observations and would let the floods dominate. Box plots, correlation and the Kolmogorov–Smirnov test use a stratified sample of up to 10,000 behaviours per network × family (187,418 rows), so every family is represented. For CICIoT2023, the analysis uses the sampled cleaned data (4.4 M windows).

4.1 Descriptive statistics

Table 7: Merged NetFlow: summary statistics of the key variables (all 17.5 M behaviours).

Variable	Mean	Median	IQR	Skewness	Max
`IN_BYTES`	3,725	701	1,459	357.2	189,027,266
`OUT_BYTES`	11,338	940	2,112	194.5	196,978,660
`IN_PKTS`	32.9	6	7	179.4	1,220,131
`FLOW_DURATION_MILLISECONDS`	14,549	755	4,010	2.4	120,999

Interpretation. Every variable is strongly **right-skewed** in both datasets (Table 7): the mean is far above the median because a few gigantic flows (up to 189 MB) pull it up, and the skewness is far above 1. In CICIoT2023, `Rate` has a mean of 14,219 packets/s but a median of 5,258 (skewness 35.8), and `IAT` has a skewness of 1,821. The **median and the IQR** therefore describe a typical flow better, and log scales are used in the plots. The NetFlow duration never exceeds 121 s because the exporter cuts longer connections.

Table 8: Merged NetFlow: median per family.

Family	`IN_BYTES`	`OUT_BYTES`	`IN_PKTS`	Duration (ms)
Benign	1,135	1,572	8	1,325
Denial of service	528	1,136	6	10,812
Reconnaissance	44	40	1	23
Credential attacks	481	2,142	6	438
Web and injection	760	1,304	6	304
Malware and botnets	288	240	6	1,506
Exploitation and intrusion	822	555	10	800
Interception and spoofing	140	258	2	37
Fuzzing and crypto	812	336	10	708
Data theft	1,658	3,380	14	2,407

Interpretation. Each family has its own typical flow (Table 8):

- **Reconnaissance** is a single probe packet (44 bytes, 1 packet);
- **Denial of service** holds connections open (median 10.8 s);
- **Data theft** moves the most data.

In CICIoT2023 the families are just as distinct:

- **DoS** windows are minimum-size packets (60 B) at about 12,000 packets/s;
- **Mirai** floods use large, uniform packets (573 B);
- **Benign** traffic has large, variable packets at a low rate.

The traffic variables therefore carry information about the attack family (SQ1).

4.2 Distributions

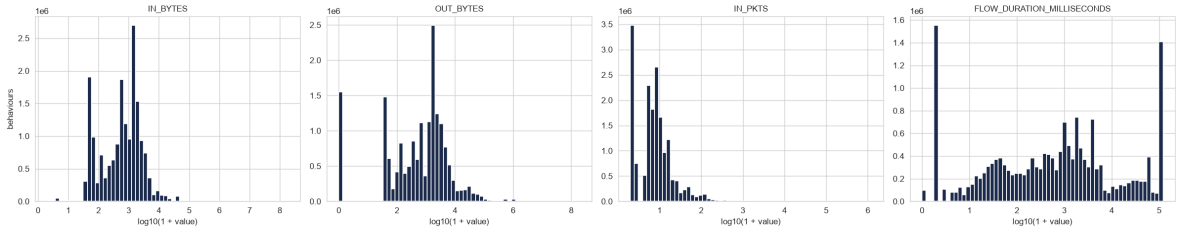


Figure 6: Merged NetFlow: key variables on a $\log_{10}(1+x)$ scale (all behaviours).

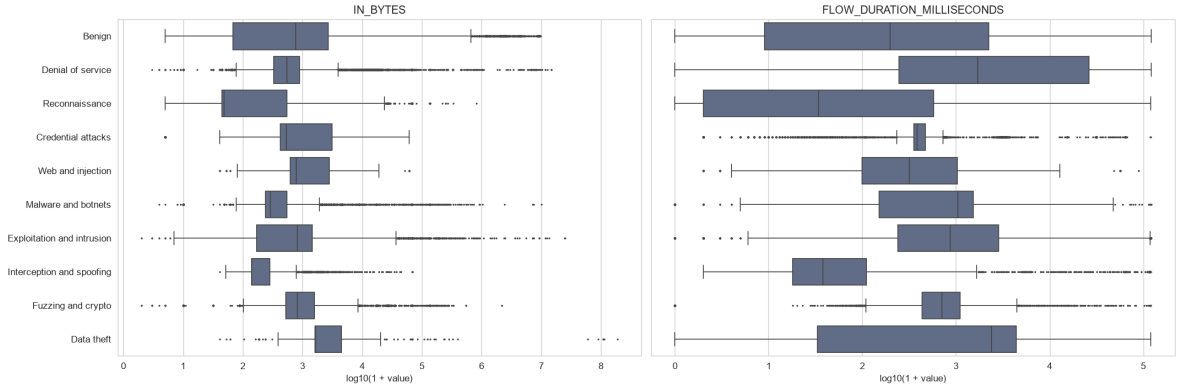


Figure 7: Merged NetFlow: bytes sent and flow duration by family (stratified sample, log scale).

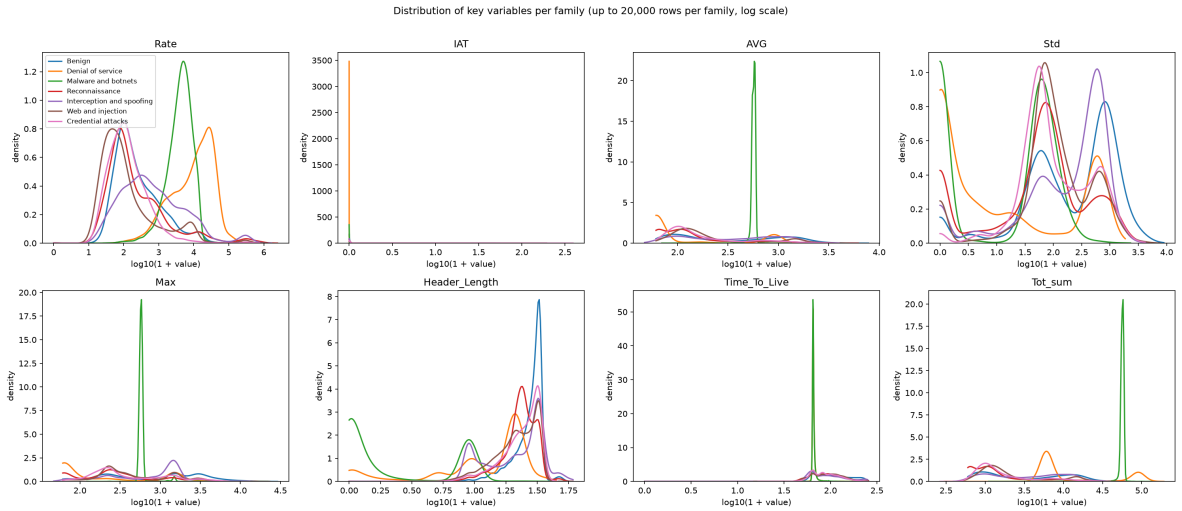


Figure 8: CICIoT2023: distributions of the key variables by family (log scale).

Interpretation. None of the variables has a bell shape (Figures 6–8); the distributions show tall spikes at technical values:

- no answer (`OUT_BYTES` $= 0$ for 8.8% of NetFlow behaviours);
- one-packet flows;
- the 120 s exporter limit;
- 1,500-byte Ethernet frames.

The families are clearly shifted from each other, but **the Benign range covers almost all attack families**: no single variable separates normal traffic from every attack, so a detector needs several variables together.

4.3 Correlation analysis

Interpretation. Pearson finds almost no relationship in NetFlow, while Spearman finds strong ones (Figure 9): for `IN_BYTES`–`IN_PKTS`, $r = 0.26$ but $\rho = 0.89$. Pearson measures straight lines and is dominated by a few gigantic flows; Spearman works on ranks, so it is the right coefficient for these skewed data. In NetFlow, bytes and packets rise together ($\rho = 0.89$), and longer flows have longer gaps between packets ($\rho = 0.84$). CICIoT2023 has 12 pairs with $|\rho| \geq 0.9$, several of them exact formulas:

- `AVG` $=$ `Tot_size`;
- `Variance` $=$ `Std`^{2};
- `ARP` $= 1 -$ `IPv`;
- `Rate` $\approx 1/$`IAT` ($\rho = -0.995$).

Such redundant columns are removed in feature selection (Section 5).

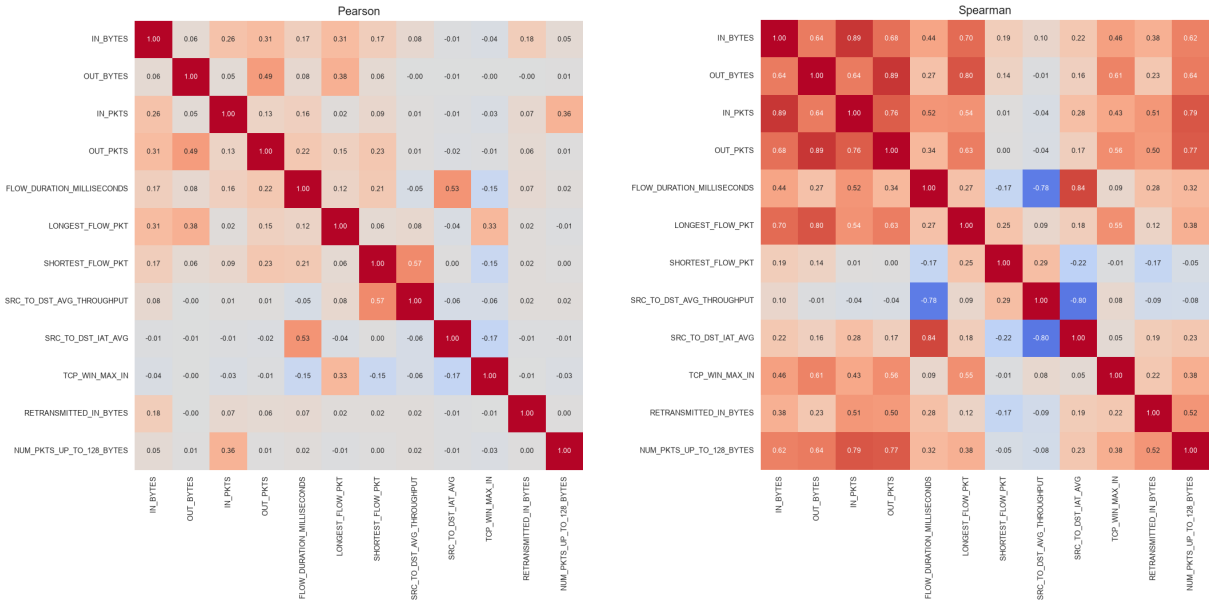


Figure 9: Merged NetFlow: Pearson (left) and Spearman (right) correlation of 12 traffic features (stratified sample).

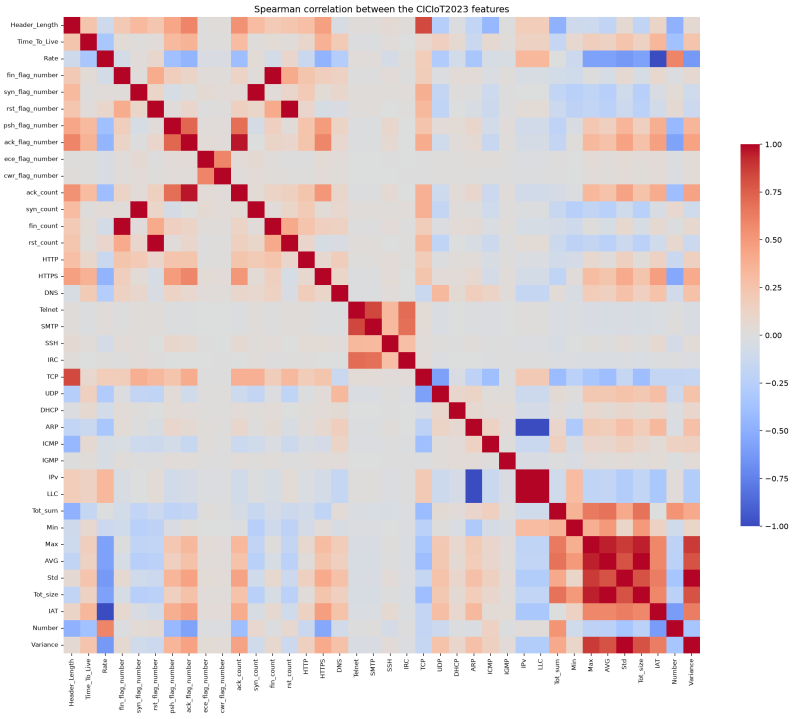


Figure 10: CICIoT2023: Spearman correlation of the window features.

4.4 Relationships between variables

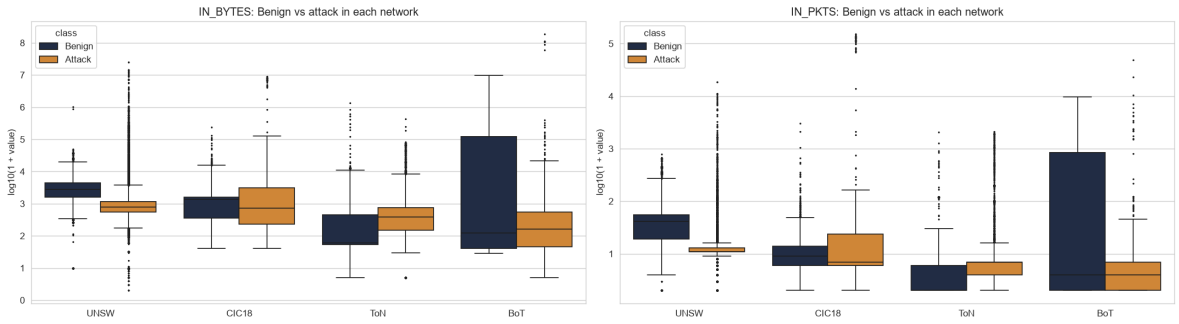


Figure 11: Merged NetFlow: bytes and packets sent, Benign against attack, in each network (stratified sample, log scale).

Table 9: Merged NetFlow: median of Benign against attack traffic in each network (all behaviours).

	IN_BYTES		IN_PKTS		Duration (ms)	
Network	Benign	Attack	Benign	Attack	Benign	Attack
UNSW	2,750	814	40	10	95	716
CIC-2018	1,369	510	8	5	2,845	3,334
ToN-IoT	65	637	1	6	22	377
BoT-IoT	120	44	3	1	31	51

Interpretation. This is the central finding (Figure 11 and Table 9): **the difference between Benign and attack traffic points in opposite directions in different networks**. In UNSW-NB15, attacks are *smaller* than normal traffic (814 vs 2,750 bytes). In ToN-IoT, they are *larger* (637 vs 65 bytes), because ToN-IoT's normal traffic consists mostly of tiny one-packet flows. Normal traffic itself differs by a factor of 40 between networks. A rule such as "attacks send more bytes" holds in one network and fails in another.

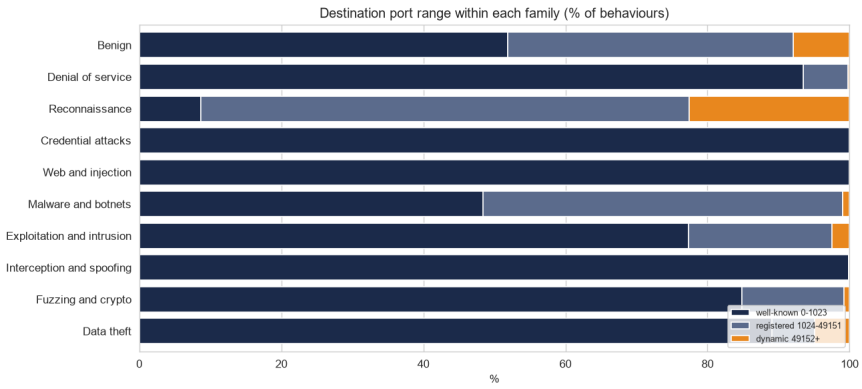


Figure 12: Merged NetFlow: destination port range within each family (% of behaviours, all data).

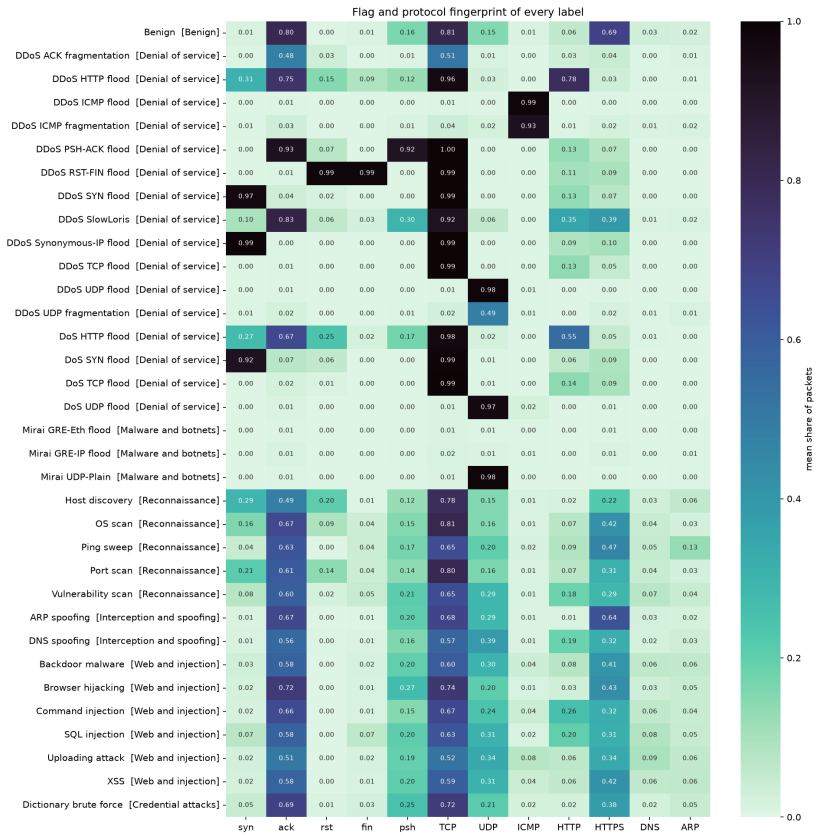


Figure 13: CICIoT2023: TCP-flag and protocol fingerprint of every attack label (share of packets).

Interpretation. The way traffic connects is strongly linked to the family (Figures 12 and 13).

- **NetFlow:** web, credential and denial-of-service traffic goes almost only to well-known ports (93–100%), while reconnaissance goes 91% to high ports, because a scan tries many ports.
- **CICIoT2023:** each flood has a fingerprint. SYN floods are about 100% SYN, PSH-ACK floods 93% PSH and ACK, ICMP floods 99% ICMP, and Mirai floods use GRE. The low-volume attacks (reconnaissance, web, credential) look like normal traffic.

4.5 Normality and the choice of tests

Interpretation. Shapiro–Wilk (on 5,000 values, its limit) and D'Agostino–Pearson reject normality (Figures 14 and 15) for **every variable, raw and log-transformed, in both datasets** (all $p < 10^{-20}$), and also inside every family. The Q-Q plots confirm that the departures are large, not a sample-size artefact. **Consequence:** the t-test and ANOVA, which assume normal groups, are not suitable, so rank-based (non-parametric) tests are used. With millions of rows every p-value is close to 0, so each test also reports an **effect size**, and conclusions rest on it. Several tests answer one question, so p-values are Bonferroni-corrected.

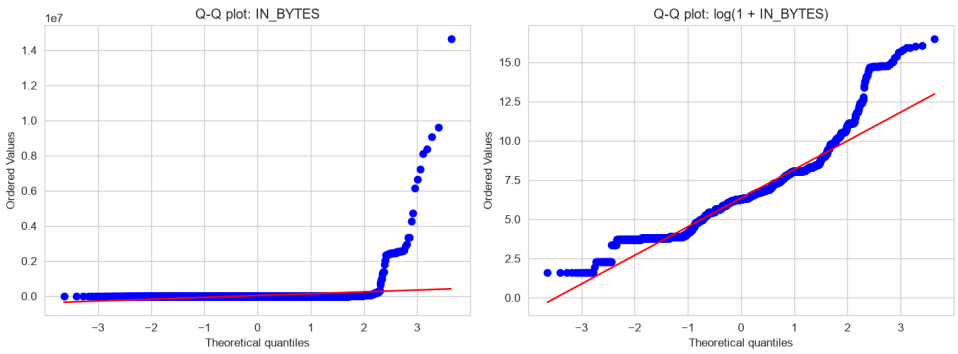


Figure 14: Merged NetFlow: Q-Q plots of `IN_BYTES` against the normal distribution, raw (left) and $\log(1+x)$ (right).

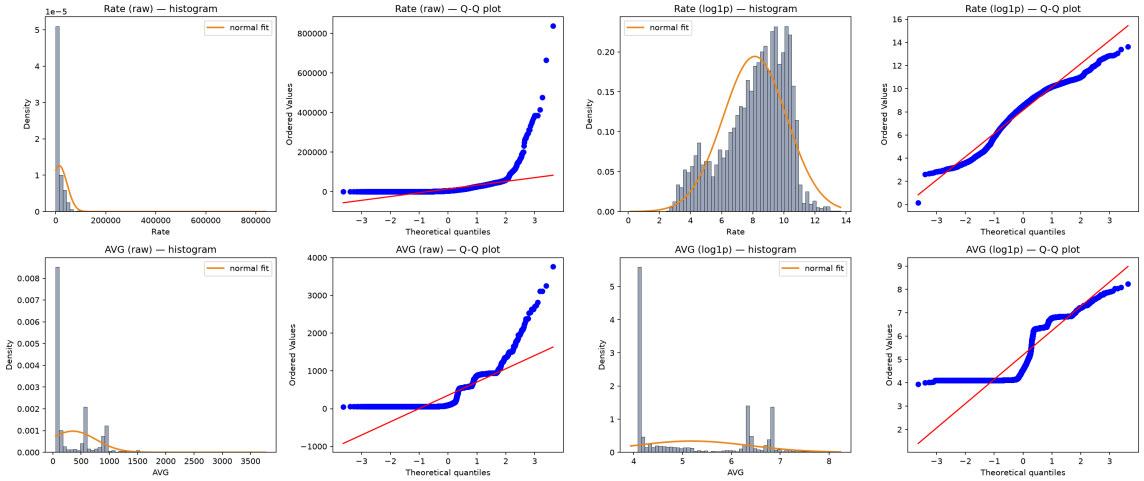


Figure 15: CICIoT2023: histograms against the normal curve and Q-Q plots of the key variables, raw and $\log(1+x)$.

4.6 Hypothesis tests

Test 1 (SQ1): Mann–Whitney U, NetFlow, attack vs Benign in each network.

- **H_0:** attack and Benign behaviours have the same distribution of the variable. **H_1:** they differ.
- **Assumptions checked and test chosen:** independent groups (one row per behaviour); normality rejected, so Mann–Whitney U instead of the two-sample t-test. All behaviours are used, which gives 16 tests.
- **Effect size:** rank-biserial $r = 2U/(n_1 n_2) - 1$; $r < 0$ means attacks are smaller. $|r| \geq 0.5$ is a large effect.

Interpretation. **Decision ($\alpha = 0.05$):** reject H_0 in all 16 tests (Table 10). The effects are large in UNSW (attacks *smaller*, r down to -0.73; a random attack flow receives more data than a random Benign flow only 14% of the time) and in ToN-IoT (attacks *larger*, r up to $+0.69$), and small in CIC-2018 and BoT-IoT. **Answer to SQ1:** attack traffic does differ from normal traffic, often strongly, but **which variable shows it and in which direction depends on the network**.

Table 10: Mann–Whitney U: effect size r of attack against Benign. All 16 tests reject H_0 (Bonferroni-corrected $p < 10^{-180}$).

Variable	UNSW	CIC-2018	ToN-IoT	BoT-IoT
IN_BYTES	−0.50	−0.11	+0.51	−0.10
OUT_BYTES	−0.73	+0.04	+0.69	−0.13
IN_PKTS	−0.62	−0.11	+0.59	−0.22
FLOW_DURATION_MILLISECONDS	+0.46	−0.05	+0.36	+0.12

Test 2 (SQ1): chi-square test of independence, CICIoT2023, protocol × family.

- **H_0:** the protocol and the attack family are independent. **H_1:** they are associated.
- **Assumptions:** independent windows (duplicates removed); expected count ≥ 5 in all cells (the 3 IGMP windows merged into "Other").
- **Effect size:** Cramér's V.

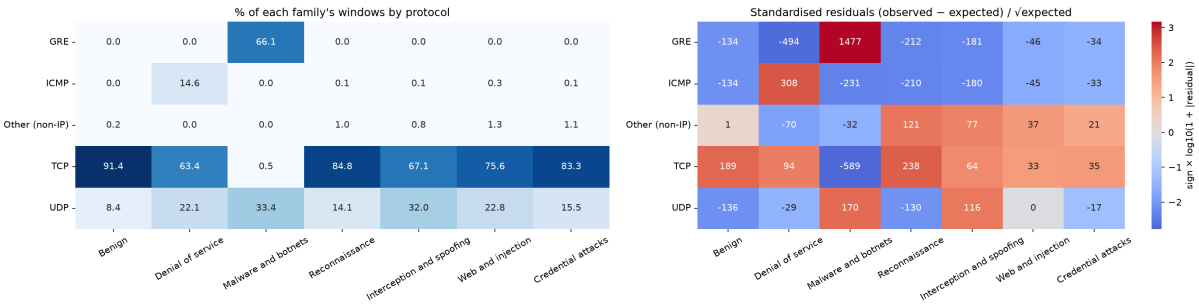


Figure 16: CICIoT2023: standardised residuals of the chi-square test, protocol × family.

Interpretation. $\chi^2(24) = 3{,}331{,}689$, $p < 0.001$. **Decision ($\alpha = 0.05$):** reject H_0. The association is strong: **Cramér's** $V = 0.43$, and 0.46 on a family-balanced sample, so it is not an artefact of the sample size. The residuals (Figure 16) show where it comes from: GRE means Mirai, ICMP means Denial of service, and non-IP windows mean reconnaissance or spoofing. In NetFlow, almost every family is mostly TCP, and the destination port range is the more informative variable.

Test 3 (SQ2): Kolmogorov–Smirnov, NetFlow, Denial of service across networks.

- **H_0:** denial-of-service flows have the same distribution in the two networks. **H_1:** they differ.
- **Assumptions and choice:** independent samples; KS compares *whole* distributions, which is what "behaving the same way" means. Denial of service is the only attack family present in all four networks: 6 pairs × 2 variables = 12 tests.
- **Effect size:** D, the largest gap between the cumulative curves (0 = identical, 1 = no overlap).

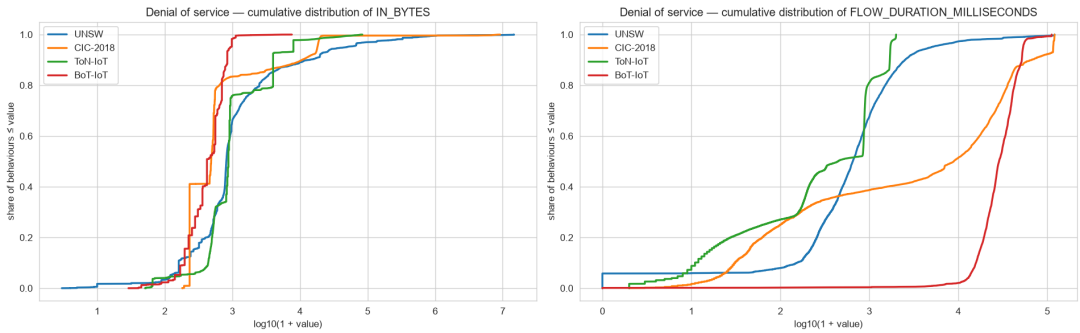


Figure 17: Denial of service: cumulative distributions of bytes and duration in the four NetFlow networks.

Interpretation. **Decision ($\alpha = 0.05$):** reject H_0 in all 12 tests (Bonferroni $p < 10^{-68}$). Nine comparisons are clearly different ($D > 0.3$), three moderately different, and none the same. A denial-of-service flow lasts a median of 0.4 s in ToN-IoT but 29 s in BoT-IoT ($D = 0.99$, Figure 17): the two distributions barely overlap. The CICIoT2023 analysis reached a related result inside one dataset: DoS and DDoS of the same flood type are practically identical (KS 0.03–0.14), so CICIoT2023 is analysed at family level.

4.7 The same attack in a different dataset: CICIoT2023 vs NetFlow

Because the two data sources cannot be merged, they are compared per family on three concepts that both measure: median packet size, packet rate, and protocol mix. A window rate and a flow rate are different measurements, so the comparison uses the **order of the families**, not the absolute values.

Table 11: Per-family profile in CICIoT2023 (windows) and in the merged NetFlow data (flows).

	CICIoT2023			NetFlow (all four)		
Family	size (B)	rate (pkt/s)	% TCP	size (B)	rate (pkt/s)	% TCP
Benign	250	171	91	159	9.7	85
Denial of service	60	**12,291**	63	152	**2.4**	87
Malware and botnets	**573**	4,403	1	**44**	7.9	99
Reconnaissance	107	122	85	42	83	100
Credential attacks	121	107	83	157	25	100
Web and injection	144	73	76	272	36	100
Interception and spoofing	359	482	67	130	103	21

Interpretation. The family profiles do not carry over between the two datasets (Table 11).

- **Denial of service is reversed:** the fastest family in CICIoT2023 (floods measured packet by packet) but the slowest in NetFlow (connections held open).
- **Malware is opposite too:** large GRE packets (Mirai) in CICIoT2023, tiny TCP packets in NetFlow.
- **Interception** is mostly TCP in CICIoT2023 but mostly UDP (DNS spoofing) in NetFlow.
- Only **reconnaissance** keeps a similar profile (small packets, TCP).

Answer to SQ2: the same attack family does **not** produce the same traffic in different networks or datasets. Tools, victims, network set-up and the measuring unit change the traffic more than the attack type keeps it alike.

4.8 Summary of the analysis

1. The data are far from normal, so medians, log scales, Spearman correlation and non-parametric tests are used.
2. Families have distinct profiles (packet size, rate, ports, flags): recognising the family looks feasible (SQ1).
3. Benign vs attack differences are real but network-specific, even reversed between networks (SQ1).
4. The same family behaves differently across networks and datasets (SQ2), so a drop in performance on an unseen network is expected (SQ3).

5 Feature Engineering

5.1 Approach

Each data source has its own feature engineering, with the same sections and rules: creation, encoding, transformation and scaling, selection ($|\rho| \geq 0.95$), final feature set, and PCA. There is **no train/test split**, because the project stops before modelling. Steps that learn from the data are fitted on all the data and must be refitted on the training part before any model, to avoid leakage.

5.2 Creation / extraction

Table 12: Created features. NetFlow: computed for all 17.5 M behaviours.

Feature	Formula	Why
NetFlow		
`in_bytes_per_pkt`, `out_bytes_per_pkt`	bytes ÷ packets	packet size, independent of flow length
`out_in_ratio`	`OUT_BYTES` ÷ `IN_BYTES`	one-way traffic vs conversation
`pkts_per_sec`, `bytes_per_sec`	volume ÷ duration	floods are fast, slow attacks slow
`no_answer`, `one_packet`	`OUT_PKTS` = 0; `IN_PKTS` = 1	closed ports, probes
`syn_only`, `hit_timeout`	client flags = SYN only; duration ≥ 120 s	half-open scans; slow attacks
CICIoT2023		
`byte_rate`, `window_duration`	`AVG` × `Rate`; `Number` × `IAT`	bytes per second; time spanned
`size_range`, `size_cv`	`Max` − `Min`; `Std` ÷ `AVG`	how uniform the packets are
`syn_minus_ack`, `control_share`	SYN share − ACK share; SYN + FIN + RST shares	half-open handshakes; control traffic
`app_share`	sum of the 7 application shares	known application vs raw flood

Interpretation. Table 12 lists the created features. In NetFlow, each family stands out in at least one of them:

- 92% of reconnaissance flows are one-packet, SYN-only probes;
- denial of service has the highest share of flows with no answer (24%);
- 68% of malware flows are SYN-only;
- credential attacks have the highest out/in ratio (4.1).

CICIoT2023 (Figure 18):

- floods send near-identical packets (`size_cv` 0.03 for denial of service, 0.08 for Mirai, against 0.61–0.74 for every other family);
- `syn_minus_ack` goes from −0.9 for Benign (mostly ACK packets) to 0.0 for the floods (no handshake);
- `control_share` is highest for reconnaissance: scans are made of connection set-up packets.

The three domain features carry more information about the family than any column they are built from.

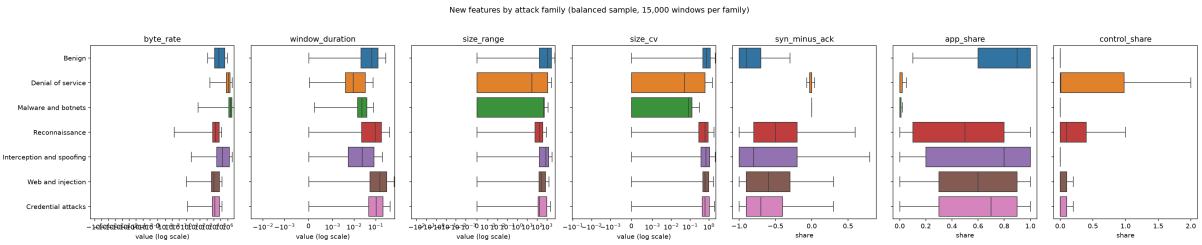


Figure 18: CICIoT2023: the new features by family.

5.3 Encoding

Code columns are identifiers, not quantities: port 443 is not "more" than port 80, and a TCP flag value of 27 means SYN + ACK + PSH + FIN.

- **NetFlow:** the 8 code columns become **32** yes/no columns:
  - destination port → 3 official ranges (well-known, registered, dynamic);
  - protocol → TCP / UDP / other;
  - client and server TCP flags → one column per flag;
  - application protocol → the 9 protocols above 1%, plus "other".
- **CICIoT2023:** `Protocol_Type` is one-hot encoded, with protocols seen fewer than 1,000 times grouped as "other". The application protocols are already numeric shares.

Alternatives considered: using the raw port number (meaningless distances); target encoding (it uses the label, so there's a risk of leakage). Binning was used only where the number itself has no meaning (the port).

5.4 Transformation and scaling

- **Transformation:**
  - NetFlow: $\log(1+x)$ on the 36 numeric columns. The number with $|\text{skewness}| \leq 1$ rises from 0 to 17; for example `IN_BYTES` goes from 107 to 1.2.
  - CICIoT2023: $\log(1+x)$ for 8 columns and Yeo-Johnson for 11. $|\text{Skewness}| \leq 1.2$ afterwards for every column except 3, which are removed in the selection; for example `Rate` goes from 62.5 to -0.6.
- **Scaling:** robust scaling, $(x - \text{median})/\text{IQR}$. The median and IQR are used instead of the mean and standard deviation because **the outliers are the attacks** and must not set the scale.
- **Special case, IQR = 0:** some columns have the same value for more than half of the rows. NetFlow has 5 (retransmissions, DNS TTL); CICIoT2023 has 2 (`fin_count`, `rst_count`, both later removed as artefacts). They are divided by their standard deviation; otherwise `RobustScaler` would leave them unscaled and they would dominate methods such as PCA. The 0/1 columns are not transformed.

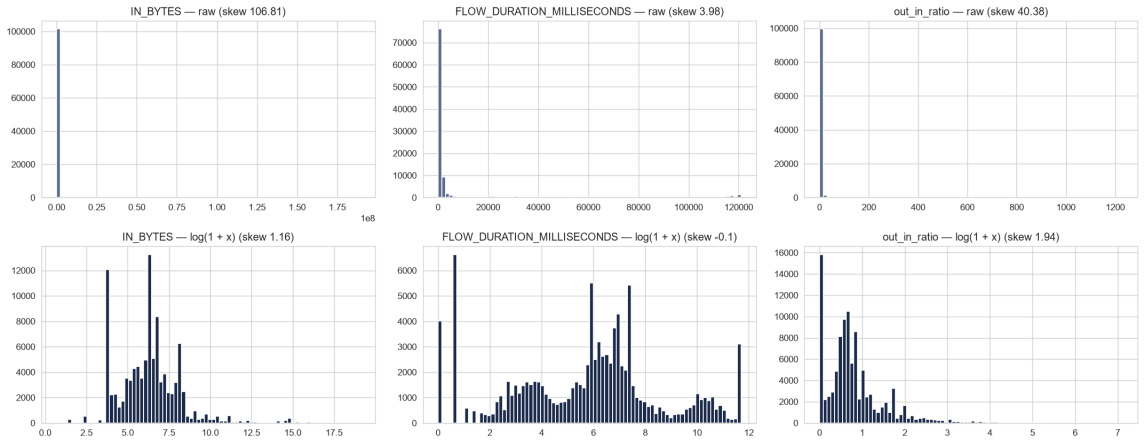


Figure 19: NetFlow: three columns before (top) and after (bottom) the log transformation.

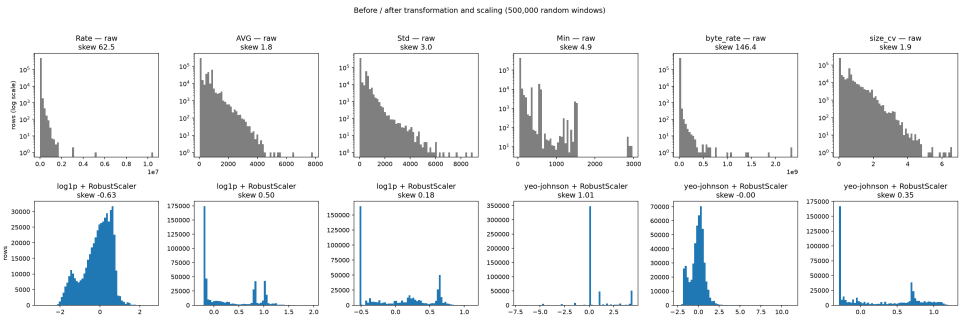


Figure 20: CICIoT2023: features before and after transformation and scaling.

5.5 Feature selection

Each removal has a reason, in five steps:

1. exact copies;
2. testbed fingerprints (columns that identify the lab, not the attack);
3. near-constant columns ($\geq 99.5\%$ one value);
4. correlated pairs ($|\rho| \geq 0.95$), keeping the column with the higher mutual information with the family;
5. a mutual information ranking.

Table 13: Selection results.

Step	NetFlow (72 candidates)	CICIoT2023
Copies	none (already removed in phase 3)	5: `Tot_size`, `Variance`, `LLC`, `IPv`, `IAT` ($\approx 1/$`Rate`)
Fingerprints	`TCP_WIN_MAX_IN`, `TCP_WIN_MAX_OUT`	8: `Number` (10- vs 100-packet capture windows), `Time_To_Live`, and 6 columns built on `Number`
Near-constant	3 rare TCP flags	5: 2 rare flags, `IRC`, `IGMP`, `proto_other`
Correlated	8 (e.g. retransmitted packets vs bytes)	2: `ack_flag_number`, `Std`
Final	**59 features**	**31 features** (from 51 candidates)

Interpretation. Table 13 summarises the selection. In NetFlow, **3 of the 5 most informative features are created ones** (`in_bytes_per_pkt`, `out_in_ratio`, `out_bytes_per_pkt`), and 8 of the 9 created features survive the selection, which justifies their creation. The TCP window is removed although it separates families well: it is set by each lab machine's operating system, so a model would learn the testbed, not the attack. **CICIoT2023:** the packet-size features lead (`Max` 0.87, `AVG` 0.80, `Header_Length` 0.68), and 4 of the top 8 features are created ones (`size_range`, `syn_minus_ack`, `size_cv`, `app_share`). The artefacts are removed even though `Tot_sum` has the highest mutual information of all (0.92): it inherits `Number`, which describes the capture method.

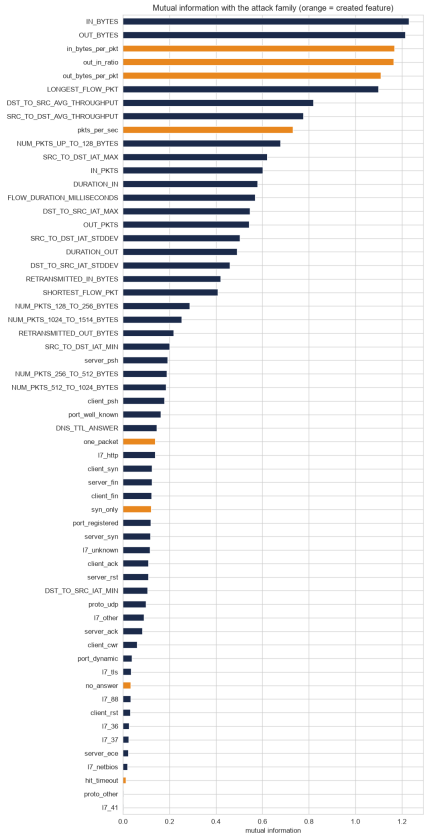


Figure 21: NetFlow: mutual information of the 59 final features with the attack family (orange = created feature).

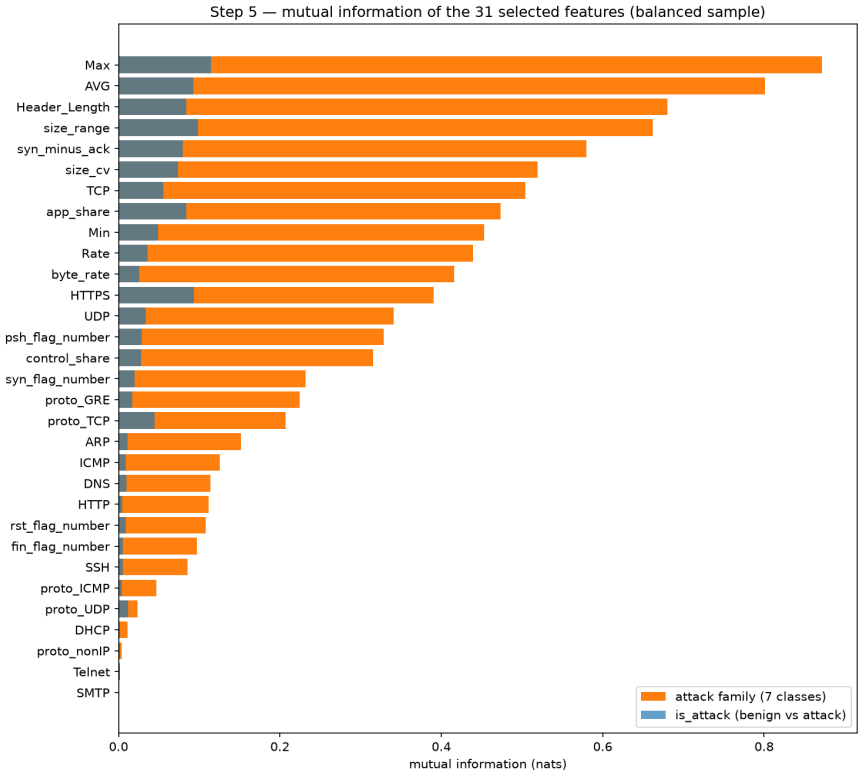


Figure 22: CICIoT2023: mutual information ranking of the final features.

Table 14: The final feature sets.

	NetFlow (merged)	CICIoT2023
Starting point	53 raw columns → 72 candidates	39 window features → 51 candidates
Original features kept	25 (volume, sizes, timing, throughput, DNS)	20 (sizes, rate, flag and protocol shares)
Created features kept	8	6
Encoded columns	26 (port range, protocol, TCP flags, application)	5 (protocol)
Total	**59**	**31**
Output	`netflow_features.parquet` (17.5 M rows)	`IoT23_features.parquet` (4.4 M rows)

5.6 Final feature sets and how they support the objectives

Interpretation. How the final sets (Table 14) support the objectives:

- **SQ1:** they keep the variables that the EDA showed to separate families (sizes, ratios, rates, ports, flags).
- **SQ3:** they contain **only traffic behaviour**, with no IP address, timestamp, TTL, TCP window or capture artefact, so a future model has to learn the attack and not the network.
- **Model-ready:** skewed numbers are log-transformed and robust-scaled, codes are one-hot encoded, and there are no duplicate columns.

Data dictionaries: `tables/NetFlow_fe_dictionary.csv` and `tables/IoT23_fe_dictionary.csv`.

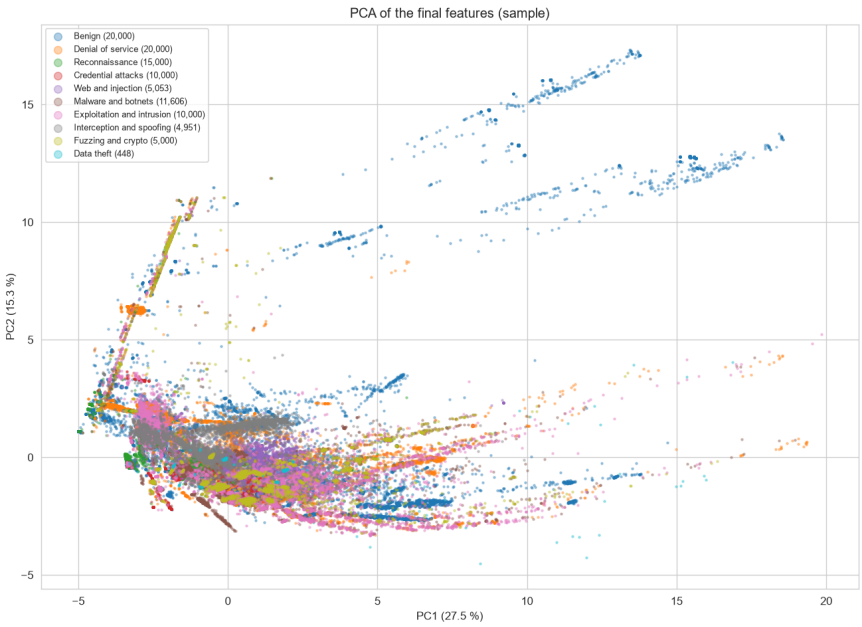


Figure 23: NetFlow: the stratified sample on the first two principal components of the 59 final features, coloured by family.

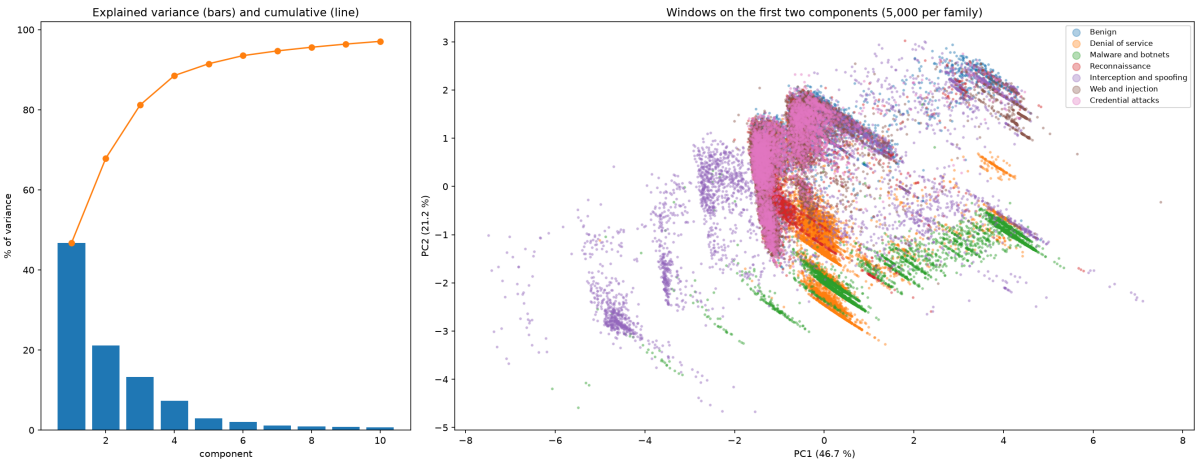


Figure 24: CICIoT2023: explained variance (left) and the first two principal components of the 31 final features (right), family-balanced sample.

Interpretation. In NetFlow (Figure 23), the first two components explain 42.8% of the variation. Reconnaissance and fuzzing form their own clusters, but most families overlap: as the EDA showed, no single direction separates the attacks, so a detector will need many features together. **CICIoT2023** (Figure 24): the first two components explain 67.9% (10 components: 97.1%). Benign, web and credential traffic lie in an upper band, while denial of service and Mirai form separate stripes below; credential attacks overlap with Benign. PC1 is dominated by `Min`, partly because robust scaling stretches its very narrow middle range, so this map depends on the scaling choice.

Techniques considered and not used:

- **hour-of-day features:** they leak the lab's attack schedule;
- **polynomial terms:** the relationships are monotonic, and the terms would add about 1,800 columns;
- **binning of volumes:** the log transform keeps more detail;
- **target encoding:** risk of leakage;
- **model-based importance:** there is no modelling in the project.

6 Conclusions and Next Steps

6.1 Answers to the research questions

Table 15 answers each research question with the evidence from Sections 2–5.

Table 15: Evidence-based answers.

SQ1	Families have clear signatures: one-packet SYN scans, long denial-of-service connections, large data-theft transfers, port and flag fingerprints (chi-square $V = 0.43$). Attack and normal traffic differ strongly inside a network (Mann–Whitney $\|r\|$ up to 0.73), but **the direction depends on the network**. No single variable separates attacks everywhere.
SQ2	**Not consistent.** The same family behaves differently across the four NetFlow networks (Kolmogorov–Smirnov, all 12 comparisons rejected, D up to 0.99) and between CICIoT2023 and NetFlow (opposite packet-rate and packet-size profiles for denial of service and malware).
SQ3	**Expectation (no model trained):** a detector will perform worse on an unseen network, because normal traffic and attacks both look different from one network to another, and some families exist in one network only. The final feature sets exclude every identifier of the network or the lab, so the loss can be measured honestly (Section 6.3).
Main	Traffic from different networks can be used to learn attack signatures, provided the data are cleaned of mislabels, identifiers and duplicated captures, and the features describe behaviour rather than the testbed. Reliability on unseen networks cannot be assumed; it must be measured.

Success criteria.

- Every sub-question is answered with tests, effect sizes and figures (criterion 1).
- Each family's indicators are identified (criterion 2).
- The final feature sets are model-ready and leakage-free (criterion 3).
- Every decision is logged and the work can be rebuilt from the repository (criterion 4, Appendix A).

6.2 Limitations

- **Testbed data:** all five datasets come from labs with scripted attacks; some differences reflect how each lab was built.
- **Coverage:** only Benign and Denial of service exist in all four NetFlow networks. Fuzzing, interception, data theft and web attacks come from one network each, so they cannot be tested on an unseen network.
- **Unit of analysis:** NetFlow statistics describe distinct behaviours, not traffic volume; CICIoT2023 uses packet windows and a cap of 200,000 rows per label.
- **Large samples:** every p-value is close to 0, so conclusions rest on effect sizes.

- **Removed labels:** CIC-2018 loses almost all its web attacks after removing the mislabelled flows.

6.3 Future modelling plan (not implemented)

1. **Split first:** 80/20, grouped by behaviour so that the same behaviour is never on both sides; refit the scaling and the selection on the training part only.
2. **Three evaluations:**
   (a) inside each network;
   (b) on all networks mixed;
   (c) **leave one network out**: train on three NetFlow networks, test on the fourth, four rounds.

   This last one answers SQ3.
3. **Imbalance:** class and network weights instead of deleting rows; realistic test sets (95% Benign / 5% attacks, and 99/1).
4. **Models:** a tree-based model (random forest or gradient boosting), which handles mixed and skewed features, compared with a simple baseline.
5. **Metrics:** macro-F1, recall per family, Benign recall and false-alarm rate, at three levels (attack or not, family, label).

References

[1] M. Luay, S. Layeghy, S. Hosseininoorbin, M. Sarhan, N. Moustafa, M. Portmann, "Temporal Analysis of NetFlow Datasets for Network Intrusion Detection Systems," arXiv:2503.04404, 2025. Data: `https://staff.itee.uq.edu.au/marius/NIDS_datasets/`.

[2] M. Sarhan, S. Layeghy, M. Portmann, "Towards a Standard Feature Set for Network Intrusion Detection System Datasets," *Mobile Networks and Applications*, vol. 27, pp. 357–370, 2022.

[3] N. Moustafa, J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," *Military Communications and Information Systems Conference (MilCIS)*, 2015.

[4] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *ICISSP*, 2018.

[5] N. Moustafa, "A New Distributed Architecture for Evaluating AI-based Security Systems at the Edge: Network TON_IoT Datasets," *Sustainable Cities and Society*, vol. 72, 102994, 2021.

[6] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, "Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.

[7] E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, no. 13, 5941, 2023. Data: `https://www.unb.ca/cic/datasets/iotdataset-2023.html`.

[8] M. Tavallaee, E. Bagheri, W. Lu, A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," *IEEE CISDA*, 2009.

[9] D. Arp et al., "Dos and Don'ts of Machine Learning in Computer Security," *USENIX Security Symposium*, 2022.

[10] P. Chapman et al., *CRISP-DM 1.0: Step-by-step Data Mining Guide*, SPSS, 2000.

[11] Software: Python 3.12, pandas, NumPy, PyArrow, SciPy, scikit-learn, matplotlib, seaborn.

A Reproducibility

- **Repository:** `https://github.com/Ahmed-Hammoda/cross-network-intrusion-analysis`.
- **Setup:** `pip install -r requirements.txt` (pandas, NumPy, PyArrow, matplotlib, seaborn, SciPy, scikit-learn, Jupyter).
- **Data:** downloaded from the official sources [1, 7] into `data/raw/`. The processed files are rebuilt by the notebooks; they are too large for the repository.
- **Shared code:** `src/config.py` (paths, taxonomy, cleaning, mapping, validation). No hard-coded paths: every notebook starts with:

```
import sys; sys.path.append("..")
from src.config import *
```

Table 16: Notebooks and run order.

Notebook	Owner	Content
`01_unsw`, `02_cic18`	A. Hamouda	phase 2: data understanding of UNSW, CIC-2018
`03_ton`, `04_bot`	C. Chabbar, A. Hamouda	phase 2: data understanding of ToN-IoT, BoT-IoT
`01–04_*_cleaning`	A. Hamouda	phase 3: cleaning of the four NetFlow datasets
`05_ciciot2023`	Y. Mtibaa	phases 2–3: CICIoT2023
`06_merge_netflow`	A. Hamouda	phase 3: merge of the NetFlow datasets
`07_eda_netflow` / `07_eda_iot23`	A. Hamouda / Y. Mtibaa	phase 4: EDA
`08_features_netflow` / `08_features_iot23`	A. Hamouda / Y. Mtibaa	phase 5: feature engineering

Run order: NetFlow cleaning (01–04) → 06 → 07 → 08; CICIoT2023: 05 → 07 → 08. About 16 GB of RAM is needed for the largest files.

B Feature dictionaries and documentation

- NetFlow v3, 53 features (meaning, unit, type, what happened to each): `docs/netflow_feature_dictionary.md`.
- CICIoT2023, 39 features: `docs/ciciot_feature_dictionary.md`.
- Final features: `tables/NetFlow_fe_dictionary.csv`, `tables/IoT23_fe_dictionary.csv`.
- Every NetFlow quality problem with evidence: `report/data_problems/phase2_data_problems.pdf`.
- Phase reports: `report/eda/phase4_eda_report.pdf`, `report/features/phase5_features_report.pdf`.
- All decisions: `docs/decision_log.md`.

C Team contributions

Member	Contribution
Ahmed Hamouda	Phase 1; data understanding of UNSW-NB15 and CSE-CIC-IDS2018; NetFlow data problems report; cleaning of the four NetFlow datasets; merge; EDA and feature engineering of the merged NetFlow data; shared code (`src/config.py`); repository and documentation.
Cyrine Chabbar	Data understanding of ToN-IoT and BoT-IoT (including the checks that found problems K17 and K18); cross-dataset comparison.
Yassine Mtibaa	CICIoT2023 through all phases: data understanding, cleaning, EDA (normality, chi-square, flag fingerprints, DoS vs DDoS), feature engineering; CICIoT2023 feature dictionary.